

Skillab

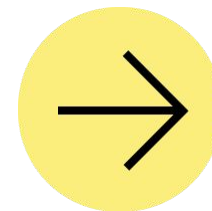
AI Agent Development

Richard Șerban Francu

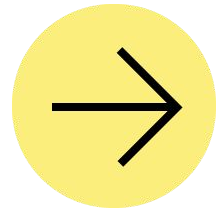
Senior AI Engineer @UCE Systems



Programa cursului



1. Noțiuni de bază — API-uri LLM și primul agent
2. Instrumente, prompturi și observabilitate
3. Procesarea și extragerea documentelor
4. Stocarea bazelor de date și RAG
5. LangGraph — Stare și fluxuri de lucru
6. **Orchestrarea multi-agent**
7. Optimizarea ML — Dincolo de apelurile LLM
8. Memorie și cache
9. Securitate și măsuri de protecție
10. Integrarea MCP
11. Inferența SageMaker
12. Training, Fine-Tuning și RLHF



Orchestrarea multi-agent

Lecția 6

Orchestrarea multi-agent

- Proiectarea arhitecturilor multi-agent (supervisor, peer-to-peer, hierarchical)
- Implementarea comunicării agent-to-agent
- Construirea agenților de planificare și orchestrare



Secțiunea 1

Multi-Agent Patterns

De ce avem nevoie de mai mulți agenți și cum îi organizăm

De Ce Multi-Agent?

User: "Procesează facturile Q1, extrage date, verifică duplicate, generează raport"

Un agent singur ar trebui să:

- Încarce și parseze documente
- Extragă date structurate
- Acceseze baza de date
- Verifice duplicatele
- Genereze raport



Rezultat

Agent supraîncărcat
Prompt enorm
Greu de debugat

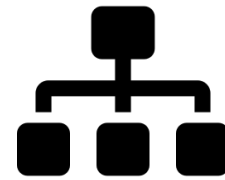
Soluția: divide et impera — fiecare agent face un singur job bine definit

Single Agent vs Multi-Agent

Aspect	Single Agent	Multi-Agent
Complexitate prompt	Un prompt imens	Prompturi mici, specializate
Debugging	Greu — totul într-un loc	Ușor — izolezi agentul problematic
Reutilizare	Dificilă	Agenți reutilizabili
Scalabilitate	Limitată	Adaugi agenți noi independent
Mentenanță	Orice schimbare afectează tot	Schimbi un agent, restul rămân

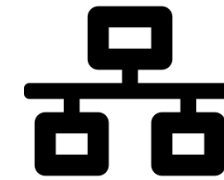
Regula de aur: Un agent = un job bine definit

Cele 3 Patterns Principale



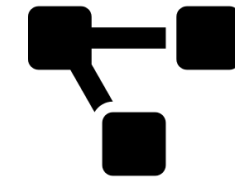
Supervisor

Un agent central coordonează și delegă la workers



Peer-to-Peer

Agenții comunică direct între ei, fără coordonator



Hierarchical

Structură pe niveluri — supervisors cu sub-supervisors



Document Analyst folosește: SUPERVISOR pattern

Pattern 1: Supervisor

Un agent central (supervisor) coordonează și delegă task-uri la agenți specializați (workers)

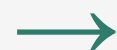
- ✓ Control centralizat
- ✓ Ușor de urmărit fluxul
- ✓ Supervisorul decide ordinea
- ✗ Single point of failure
- ✗ Bottleneck la supervisor

Când folosești:

- Task-uri cu pași secvențiali
- Ai nevoie de un orchestrator
- Vrei control asupra fluxului

Document Analyst — Supervisor Pattern

ANALYST



Extractor 1

Extractor 2

Extractor 3

Pattern 2: Peer-to-Peer

Agenții comunică direct între ei, fără un coordonator central

- ✓ Fără bottleneck
- ✓ Fault tolerant
- ✓ Flexibil și dinamic
- ✗ Greu de urmărit fluxul
- ✗ Potențial cicluri infinite
- ✗ Sincronizare complexă

Când folosești:

- Sisteme conversaționale multi-personaj
- Agenți care colaborează
- Nu ai nevoie de control strict

Exemplu (ipotetic Document Analyst):

Extractor

→ trimite chunk →

Validator

→ feedback →

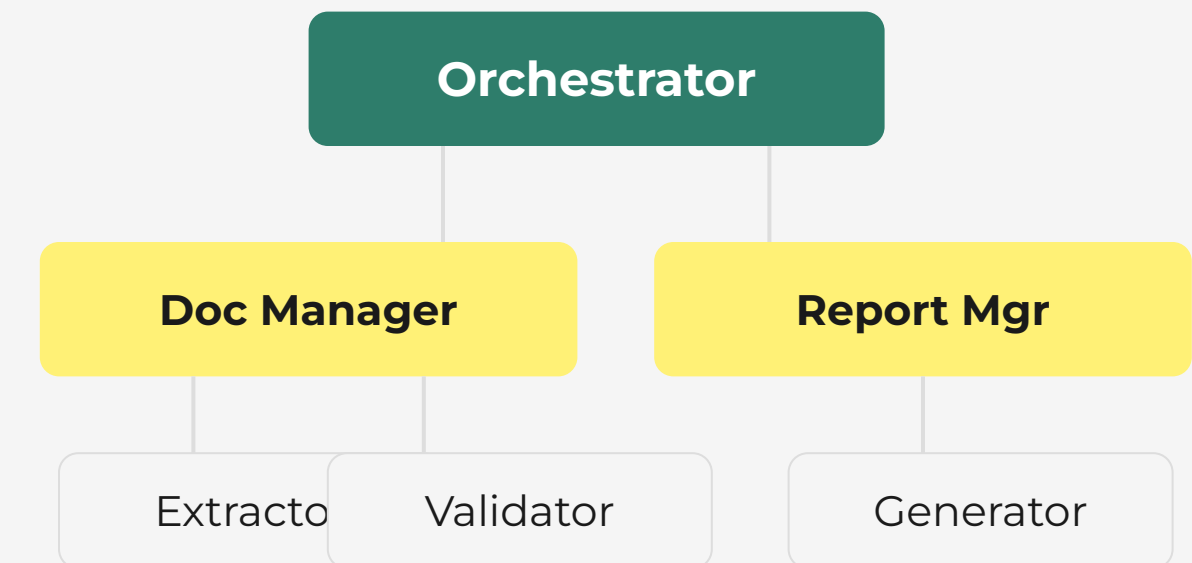
Extractor

Pattern 3: Hierarchical

Structură pe mai multe niveluri — supervisors care au sub-supervisors

- ✓ Scalează la sisteme complexe
- ✓ Separare clară responsabilități
- ✓ Decizii la nivelul potrivit
- ✗ Complexitate ridicată
- ✗ Latență mai mare
- ✗ Overkill pentru task-uri simple

Document Analyst extins:



Când folosești:

Organizații mari cu multiple echipe • Sub-task-uri cu sub-sub-task-uri • Enterprise workflows

Comparație Patterns

Criteriu	Supervisor	Peer-to-Peer	Hierarchical
Complexitate	Medie	Ridicată	Foarte ridicată
Control	Centralizat	Distribuit	Multi-nivel
Debugging	Ușor	Greu	Mediu
Latență	Medie	Scăzută	Ridicată
Scalabilitate	Medie	Bună	Foarte bună
Use case	Workflows	Colaborare	Enterprise

Pentru Document Analyst → SUPERVISOR

Task clar definit • Pași secvențiali (plan → extract → aggregate) • Debugging ușor

Supervisor în LangGraph

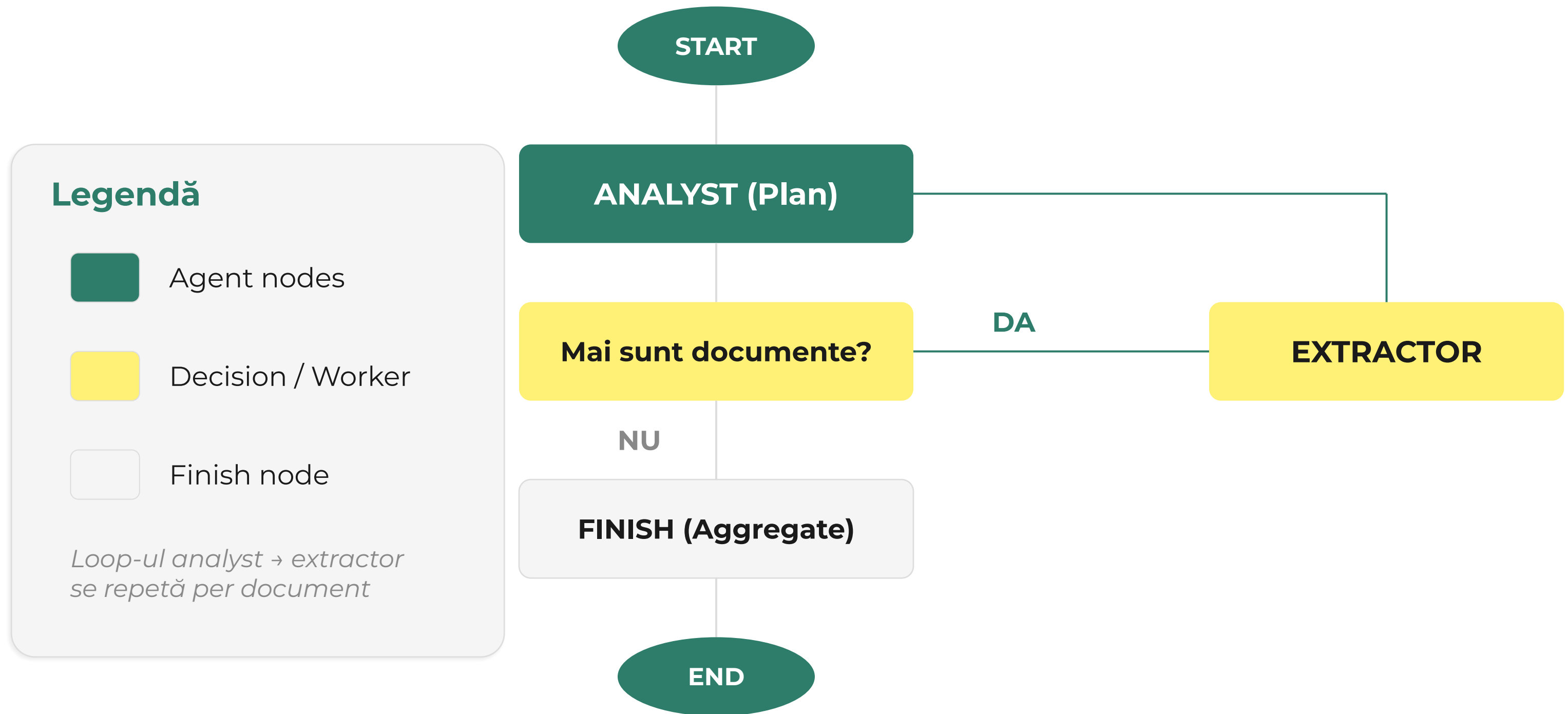
LangGraph oferă primitive native pentru multi-agent:

```
1 class ExtractedData(BaseModel):
2     source: str
3     fields: dict[str, str]
4
5
6 # 1. State-ul partajat
7 class AnalystState(TypedDict):
8     documents: list[str]
9     results: list[ExtractedData]
10    current_doc: int
11
12
13 class DocumentAnalystAgent:
14     # __init__, noduri, routere ...
15
16     # 2. Supervisor node
17     def analyst_node(self, state: AnalystState) -> AnalystState:
18         # call LLM here pentru planning / decizie
19         return state
20
21     # 3. Worker node
22     def extractor_node(self, state: AnalystState) -> AnalystState:
23         # call LLM here cu structured output -> ExtractedData
24         return {**state, "current_doc": state["current_doc"] + 1}
25
26     # 4. Router - decide direcția
27     def should_continue(self, state: AnalystState) -> Literal["extractor", "finish"]:
28         if state["current_doc"] < len(state["documents"]):
29             return "extractor"
30         return "finish"
```

Graf Supervisor în LangGraph

```
1 class DocumentAnalystAgent:
2     # __init__, noduri, routere ...
3
4     def build_graph(self):
5         # 1. Schema
6         graph = StateGraph(AnalystState)
7
8         # 2. Noduri
9         graph.add_node("analyst", self.analyst_node)
10        graph.add_node("extractor", self.extractor_node)
11        graph.add_node("finish", self.finish_node)
12
13        # 3. Edge de start
14        graph.add_edge(START, "analyst")
15
16        # 4. Edges simple
17        graph.add_edge("extractor", "analyst")
18        graph.add_edge("finish", END)
19
20        # 5. Conditional edges
21        graph.add_conditional_edges(
22            "analyst",
23            self.should_continue,
24            {"extractor": "extractor", "finish": "finish"},
25        )
26
27        # 6. Compile
28        return graph.compile()
29
```

Flux Document Analyst

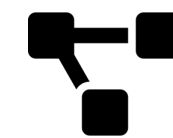


Sumar Secțiunea 1



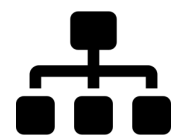
De ce multi-agent

Divide et impera — un agent = un job bine definit



3 patterns

Supervisor (control)
Peer-to-Peer (colaborare)
Hierarchical (enterprise)



Document Analyst

Supervisor pattern:
Analyst = coordonator
Extractor = worker



LangGraph

StateGraph cu state partajat
Conditional edges
Noduri ca funcții

Next: Secțiunea 2 — Agent Communication Protocols



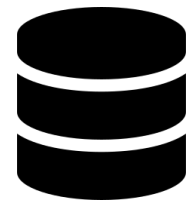
Secțiunea 2

Agent Communication

Shared State, Message Passing și Tool Calling

Cum Comunică Agenții?

Când Analyst-ul delegă un task la Extractor, cum transmite informațiile?



Shared State

Agenții citesc/scriu
acelaș state

`TypedDict` partajat



Message Passing

Agenții își trimit
mesaje

`messages în state`



Tool Calling

Un agent apelează alt
agent ca tool

`@tool decorator`

Document Analyst: Shared State + Message Passing

Shared State: Conceptul

Toți agenții au acces la același obiect de state și îl modifică

```
SHARED STATE
{
  documents: ["doc1.pdf", "doc2.pdf"],
  plan: [...],
  results: [...],
  current_doc: 1
}
```

Analyst

read/write

Extractor

read/write

Reporter

read only

Avantaje

- ✓ Simplu de implementat
- ✓ Tot contextul e disponibil
- ✓ Natural în LangGraph

Shared State în LangGraph

```
from typing import TypedDict, Annotated
from langgraph.graph import add_messages

class AnalystState(TypedDict):
    """State partajat între agenți"""

    # Input
    documents: list[str]
    query: str

    # Plan (Analyst scrie, Extractor citește)
    plan: list[dict]

    # Rezultate – reducer add = append!
    results: Annotated[list, operator.add]

    # Control flow
    current_doc_index: int
    status: str # planning | extracting | done

    # Mesaje pentru comunicare
    messages: Annotated[list, add_messages]
```

Reducers: Cum Se Combină Updates

Problema:

```
1 # Extractor 1 returnează:
2 {"results": [ExtractedData(source="doc1.pdf", fields={})]}
3
4 # Extractor 2 returnează:
5 {"results": [ExtractedData(source="doc2.pdf", fields={})]}
6
7
```

✗ **Fără reducer:**
ultimul câștigă → [result_2]

✓ **Cu reducer add:**
se combină → [result_1, result_2]

```
1 # Soluția: reducer-i pe câmpurile din state
2 class State(TypedDict):
3     # Fără reducer – replace
4     current_doc: str
5
6     # Cu reducer – append
7     results: Annotated[list[ExtractedData], operator.add]
8
9     # Reducer pentru mesaje – append + deduplicare după ID
10    messages: Annotated[list[AnyMessage], add_messages]
11
```

add_messages = deduplicare automată + append (nu duplică mesaje cu același ID)

Message Passing: Conceptul

Agentii comunică prin mesaje într-o listă partajată

MESSAGES LIST

Human: "Procesează facturile Q1"

AI/Analyst: "Am creat planul cu 3 documente"

AI/Extractor: "Doc 1: Extras factura #2024-001"

AI/Extractor: "Doc 2: Extras factura #2024-002"

AI/Analyst: "Finalizat. Raport generat."

Fiecare agent:

Citește istoricul de mesaje pentru context • Adaugă propriile mesaje • Nu șterge mesajele altor agenți

Message Passing în LangGraph

```
1 def analyst_node(self, state: AnalystState) -> dict:
2     """Analyst citește mesajele, adaugă planul."""
3     messages = self._to_provider_messages(state["messages"])
4     response = self.llm.generate_sync(messages)
5     plan = self._parse_plan(response)
6
7     return {
8         "messages": [
9             AIMessage(
10                 content=f"Am creat planul cu {len(plan)} documente",
11                 name="analyst",
12             )
13         ],
14         "plan": plan,
15     }
16
17 def extractor_node(self, state: AnalystState) -> dict:
18     """Extractor procesează și raportează."""
19     doc = state["documents"][state["current_doc_index"]]
20     result = self._extract_document(doc)
21
22     return {
23         "messages": [
24             AIMessage(
25                 content=f"Procesat {doc}: {result.source}",
26                 name="extractor",
27             )
28         ],
29         "results": [result], # append via reducer
30     }
31
32 @staticmethod
33 def _to_provider_messages(messages: list[AnyMessage]) -> list[Message]:
34     role_map = {"human": "user", "ai": "assistant", "system": "system"}
35     return [{"role": role_map[m.type], "content": m.content} for m in messages]
36
```

Agent ca Tool (Tool Calling)

Alternativă: un agent poate apela alt agent ca pe un tool

```
1 def call_extractor(self, document_path: str) -> ExtractedData:
2     """Apelează Extractor agent (sub-graph) ca pe un tool."""
3     result = self.extractor_graph.invoke({"document": document_path})
4     return result["extracted_data"]
5
6 def analyst_node(self, state: AnalystState) -> dict:
7     """Analyst decide via prompt dacă apelează un tool sau răspunde direct."""
8     messages = self._to_provider_messages(state["messages"])
9     response = self.llm.generate_sync(messages)
10
11     tool_call = self._parse_tool_call(response)
12     if tool_call is None:
13         return {"messages": [AIMessage(content=response, name="analyst")]}
14
15     tool_result = self.tools[tool_call.name](*tool_call.args)
16     return {
17         "messages": [
18             AIMessage(
19                 content=f"Tool {tool_call.name} -> {tool_result.source}",
20                 name="analyst",
21             )
22         ],
23         "results": [tool_result],
24     }
25
26 @staticmethod
27 def _parse_tool_call(response: str) -> ToolCall | None:
28     try:
29         return ToolCall.model_validate_json(response)
30     except ValidationError:
31         return None
```

Document Analyst: folosim Shared State + Messages (mai simplu). Tool Calling pentru extensii viitoare.

Când folosești:

- ✓ Extractor e un sistem complex separat
- ✓ Vrei izolare completă între agenți
- ✓ Ai nevoie de API clar

Comparație Metode de Comunicare

Aspect	Shared State	Message Passing	Tool Calling
Complexitate	Scăzută	Medie	Ridicăta
Coupling	Strâns	Mediu	Slab
Debugging	Ușor (vezi state)	Mediu (citești msgs)	Greu (call stack)
Flexibilitate	Medie	Bună	Foarte bună
Use case	Workflows simple	Conversații	Sisteme distribuite

Document Analyst

Shared State pentru date (documents, results, plan) **Messages** pentru comunicare și logging

Pattern: Command Objects

Pentru task-uri structurate, folosim comenzi tipizate (Pydantic):

ExtractCommand

```
1 class ExtractCommand(BaseModel):
2     model_config = ConfigDict(frozen=True)
3
4     action: Literal["extract"] = "extract"
5     document_path: str = Field(min_length=1)
6     schema_type: SchemaType
7     priority: int = Field(default=1, ge=1, le=10)
8
9
```

ExtractResult

```
1 class ExtractResult(BaseModel):
2     model_config = ConfigDict(frozen=True)
3
4     document_path: str = Field(min_length=1)
5     status: Literal["success", "error"]
6     data: ExtractedData | None = None
7     error_message: str | None = None
8
```

Avantaje:

Validare automată (Pydantic) • Tipuri clare între agenți • Ușor de testat și debugat

Error Communication

Cum raportează Extractor-ul o eroare către Analyst?

```
1 class ExtractionError(Exception):
2     """Eroare la extracția datelor dintr-un document."""
3
4
5 class ExtractorAgent:
6     def __init__(self, llm: LLMPProvider):
7         self.llm = llm
8
9     def extractor_node(self, state: AnalystState) -> dict:
10         doc = state["documents"][state["current_doc_index"]]
11
12         try:
13             data = self._extract_document(doc)
14             result = ExtractResult(document_path=doc, status="success", data=data)
15         except ExtractionError as e:
16             logger.warning("Extraction failed for %s: %s", doc, e)
17             result = ExtractResult(document_path=doc, status="error", error_message=str(e))
18
19         return {"results": [result]}
```



Analyst decide:

Retry documentul?

Skip și continuă?

Abort totul?

Structură consistentă success/error → Analyst poate lua decizii inteligente (→ Secțiunea 5)

Sumar — Secțiunea 2

3 metode

Shared State (simplu)
Message Passing (flexibil)
Tool Calling (decuplat)

Reducers

Annotated[list, add] → append
add_messages → deduplicare
Fără reducer → replace

Command Objects

Pydantic models
Validare automată
Tipuri clare între agenți

Error Communication

Structură success/error
Analyst decide: retry/skip/abort
→ Secțiunea 5 detalii

Next: Secțiunea 3 — Building the Analyst Agent



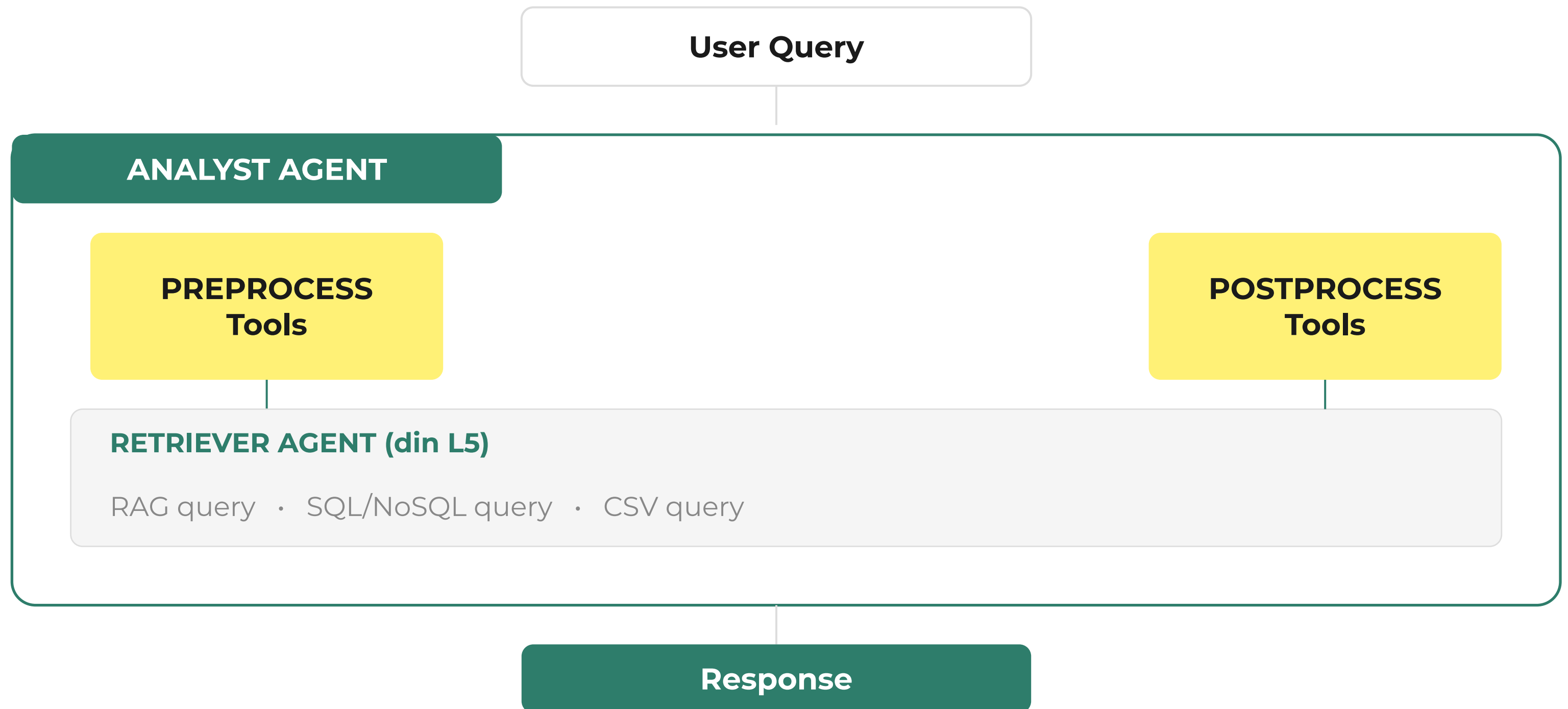
Secțiunea 3

Building the Analyst Agent

Preprocess + Postprocess tools pentru orchestrare

Arhitectura Runtime

RECAP: L3-4: Extractor → RAG/SQL/NoSQL/CSV · L5: Retriever scrie query-uri · **L6: Analyst orchestrează**



Rolul Analyst Agent

Analyst = Orchestrator cu Preprocess + Postprocess

Fază	Ce face	Tools
Preprocess	Pregătește query pentru Retriever	parse_query, detect_intent, extract_filters
Delegate	Apelează Retriever Agent	call_retriever
Postprocess	Procesează datele primite	aggregate, filter, format, summarize

Analyst NU:

- ❌ Nu accesează direct data stores
- ❌ Nu scrie query-uri SQL/RAG
- ❌ Nu extrage date din documente

Analyst DA:

- ✅ Înțelege query-ul utilizatorului
- ✅ Pregătește ce să ceară de la Retriever
- ✅ Procesează și formatează răspunsul

System Prompt pentru Analyst



```
1  name: analyst_system
2  version: 1.0.0
3  description: >
4      System prompt pentru Analyst Agent în Document Analyst.
5      Orchestrează Retriever Agent pentru acces la RAG / SQL / CSV.
6  prompt: |
7      # IDENTITATE
8      Ești Analyst Agent în sistemul Document Analyst.
9      Rolul tău: orchestrezi Retriever Agent.
10
11     # CONTEXT
12     Date extrase din documente, stocate în:
13     - RAG (căutări semantice)
14     - SQL/NoSQL (date structurate)
15     - CSV (export/rapoarte)
16     NU accesezi direct. Folosești Retriever Agent.
17
18     # INSTRUCȚIUNI
19     1. PREPROCESS: analizează query-ul (intent, filters)
20     2. DELEGATE: apelează Retriever cu query pregătit
21     3. POSTPROCESS: agregă, filtrează, formatează
22
23     # CONSTRÂNGERI
24     - NU inventa date
25     - La eroare de la Retriever → raportează clar
26
```

Preprocess Tools: Overview

User: "Care sunt totalurile facturilor de la TechSoft din Q1?"



PREPROCESS PIPELINE

 **parse_query**

```
"facturi"  
"TechSoft"  
"Q1"  
"totaluri"
```

 **detect_intent**

```
intent:  
"aggregate"
```

 **extract_filters**

```
supplier: "TechSoft"  
period: Q1  
field: total
```

Output: PreparedQuery pentru Retriever

Tool: parse_query

```
1  class ParsedQuery(BaseModel):
2      """Rezultatul parsării query-ului."""
3
4      entities: list[str] # furnizori, date
5      keywords: list[str] # cuvinte cheie
6      doc_types: list[str] # tipuri docs
7      time_references: list[str] # referințe temporale
8
9
10 @register_tool
11 def parse_query(params: ParseQueryParams) -> str:
12     """Parsează query-ul și extrage componente.
13
14     Folosește pentru: prima etapă de preprocess.
15     """
16     parsed = ParsedQuery(
17         entities=extract_entities(params.query),
18         keywords=extract_keywords(params.query),
19         doc_types=detect_doc_types(params.query),
20         time_references=extract_time_refs(params.query),
21     )
22     return parsed.model_dump_json()
```

Tool: detect_intent

```
1 Intent = Literal["search", "aggregate", "compare", "filter", "export"]
2 Source = Literal["rag", "sql", "nosql", "csv"]
3
4
5 class QueryIntent(BaseModel):
6     intent: Intent
7     confidence: float = Field(ge=0.0, le=1.0)
8     suggested_source: Source
9
10
11 INTENT_RULES: dict[Intent, tuple[list[str], Source]] = {
12     "aggregate": (["total", "sumă", "câte", "număr", "media"], "sql"),
13     "compare": (["compară", "vs", "diferență", "față de"], "sql"),
14     "filter": (["unde", "doar", "filtrează", "cu"], "nosql"),
15     "export": (["export", "descarcă", "csv", "raport"], "csv"),
16     "search": (["caută", "găsește", "arată", "ce"], "rag"),
17 }
18
19 DEFAULT_INTENT: Intent = "search"
20 DEFAULT_SOURCE: Source = "rag"
21 DEFAULT_CONFIDENCE = 0.3
22
23
24 @register_tool
25 def detect_intent(params: ParseQueryParams) -> str:
26     """Detectează intent-ul query-ului și sugerează sursa de date.
27
28     Folosește pentru: a doua etapă de preprocess, după parse_query.
29     """
30     q = params.query.lower()
31
32     for intent, (keywords, source) in INTENT_RULES.items():
33         if any(w in q for w in keywords):
34             result = QueryIntent(intent=intent, confidence=0.9, suggested_source=source)
35             return result.model_dump_json()
36
37     fallback = QueryIntent(
38         intent=DEFAULT_INTENT,
39         confidence=DEFAULT_CONFIDENCE,
40         suggested_source=DEFAULT_SOURCE,
41     )
42     return fallback.model_dump_json()
43
```



Intents recunoscuți:

search

căutare docs

aggregate

sum, count,
avg

compare

comparație

filter

filtrare

export

export date

Tool: extract_filters

```
1  @register_tool
2  def set_date_range(params: SetDateRangeParams) -> str:
3      """Setează intervalul de date pentru filtrare."""
4      _filters.date_from = params.date_from.isoformat()
5      _filters.date_to = params.date_to.isoformat()
6      return _filters.model_dump_json()
7
8
9  @register_tool
10 def add_suppliers(params: AddSuppliersParams) -> str:
11     """Adaugă furnizori la lista de filtrare."""
12     _filters.suppliers = list({*_filters.suppliers, *params.suppliers})
13     return _filters.model_dump_json()
14
15
16 @register_tool
17 def add_doc_types(params: AddDocTypesParams) -> str:
18     """Adaugă tipuri de documente la lista de filtrare."""
19     _filters.doc_types = list({*_filters.doc_types, *params.doc_types})
20     return _filters.model_dump_json()
21
22
23 @register_tool
24 def set_amount_range(params: SetAmountRangeParams) -> str:
25     """Setează intervalul de valori monetare."""
26     _filters.amount_min = params.amount_min
27     _filters.amount_max = params.amount_max
28     return _filters.model_dump_json()
29
```

Postprocess Tools: Overview

Retriever returnează:

```
[{"doc": "fact_001", "total": 5000, "supplier": "TechSoft"},  
 {"doc": "fact_002", "total": 3500, ...}, ...]
```



POSTPROCESS PIPELINE



aggregate

sum: 16700

count: 3

avg: 5566



filter

(remove
nulls)



format

"Total Q1:

16,700 RON"

Output: Răspuns final formatat pentru user

Tool: aggregate_results

```
1 class AggregateParams(BaseModel):
2     data: str # JSON array de la Retriever
3     operations: list[Operation] = Field(min_length=1)
4     field: str = Field(min_length=1)
5     group_by: str | None = None
6
7
8 @register_tool
9 def aggregate_results(params: AggregateParams) -> str:
10     """Agregă datele: sum, count, avg, min, max. Suportă group_by."""
11     items: list[dict[str, Any]] = json.loads(params.data)
12
13     if params.group_by is None:
14         return json.dumps(_aggregate(items, params.field, params.operations))
15
16     groups: dict[Any, list[dict[str, Any]]] = {}
17     for item in items:
18         key = item.get(params.group_by)
19         if key is None:
20             continue
21         groups.setdefault(key, []).append(item)
22
23     grouped = {
24         str(key): _aggregate(group, params.field, params.operations)
25         for key, group in groups.items()
26     }
27     return json.dumps(grouped)
28
```

Tool: format_response

```
1 class FormatResponseParams(BaseModel):
2     data: str
3     format_type: Literal["text", "table", "summary", "list"]
4     query: str = Field(min_length=1)
5     language: str = "ro"
6
7
8 @register_tool
9 def format_response(params: FormatResponseParams) -> str:
10     """Formatează datele final pentru utilizator."""
11     data = json.loads(params.data)
12
13     formatters = {
14         "summary": generate_summary,
15         "table": format_as_table,
16         "list": format_as_list,
17         "text": format_as_text,
18     }
19     return formatters[params.format_type](data, params.query, params.language)
20
```



Format types

summary

"Totalul: 16,700 RON"

table

col val

list

- item 1
- item 2

text

paragraf liber

AnalystAgent: Clasa Completă

```
1 class AnalystAgent:
2     """Agent Analyst cu preprocess și postprocess tools."""
3
4     def __init__(self, provider: str = "ollama"):
5         self.provider = provider
6         self.llm = self._create_llm(provider)
7         self.system_prompt = ANALYST_SYSTEM_PROMPT
8         self.tool_wrapper = ToolWrapper()
9
10    def process(self, query: str) -> str:
11        """Query → preprocess → retrieve → postprocess."""
12        # 1. PREPROCESS
13        prepared = self._preprocess(query)
14        # 2. RETRIEVE
15        raw_data = self._retrieve(prepared)
16        # 3. POSTPROCESS
17        response = self._postprocess(raw_data, prepared, query)
18        return response
```

Flow Complet: Exemplu

```
query = "Care sunt totalurile facturilor TechSoft din Q1 2024?"
```

1. PREPROCESS

```
parse_query → entities: ["TechSoft"], time: ["Q1 2024"]  
detect_intent → intent: "aggregate", source: "sql"  
extract_filters → supplier: "TechSoft", date: Q1 2024
```

2. RETRIEVE

```
call_retriever → Retriever scrie SQL:  
SELECT * FROM invoices WHERE supplier='TechSoft'  
AND date BETWEEN '2024-01-01' AND '2024-03-31'  
→ [{"total": 5000}, {"total": 3500}, {"total": 8200}]
```

3. POSTPROCESS

```
aggregate_results → {"sum": 16700, "count": 3}  
format_response → "Totalul facturilor TechSoft Q1 2024:  
16,700 RON (3 facturi)"
```

Sumar — Secțiunea 3



Rolul Analyst

Orchestrator pur:
NU accesează data stores
Delegă la Retriever



Preprocess Tools

parse_query
detect_intent
extract_filters



Postprocess Tools

aggregate_results
filter_results
format_response



Flow Complet

Query → Preprocess
→ Retrieve → Postprocess
→ Response

Next: Secțiunea 4 — Orchestration Flow (LangGraph)



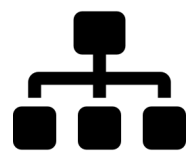
Secțiunea 4

Orchestration Flow

LangGraph workflow pentru Document Analyst

Ce Este Orchestration?

Coordonarea execuției mai multor agenți pentru a îndeplini un task complex



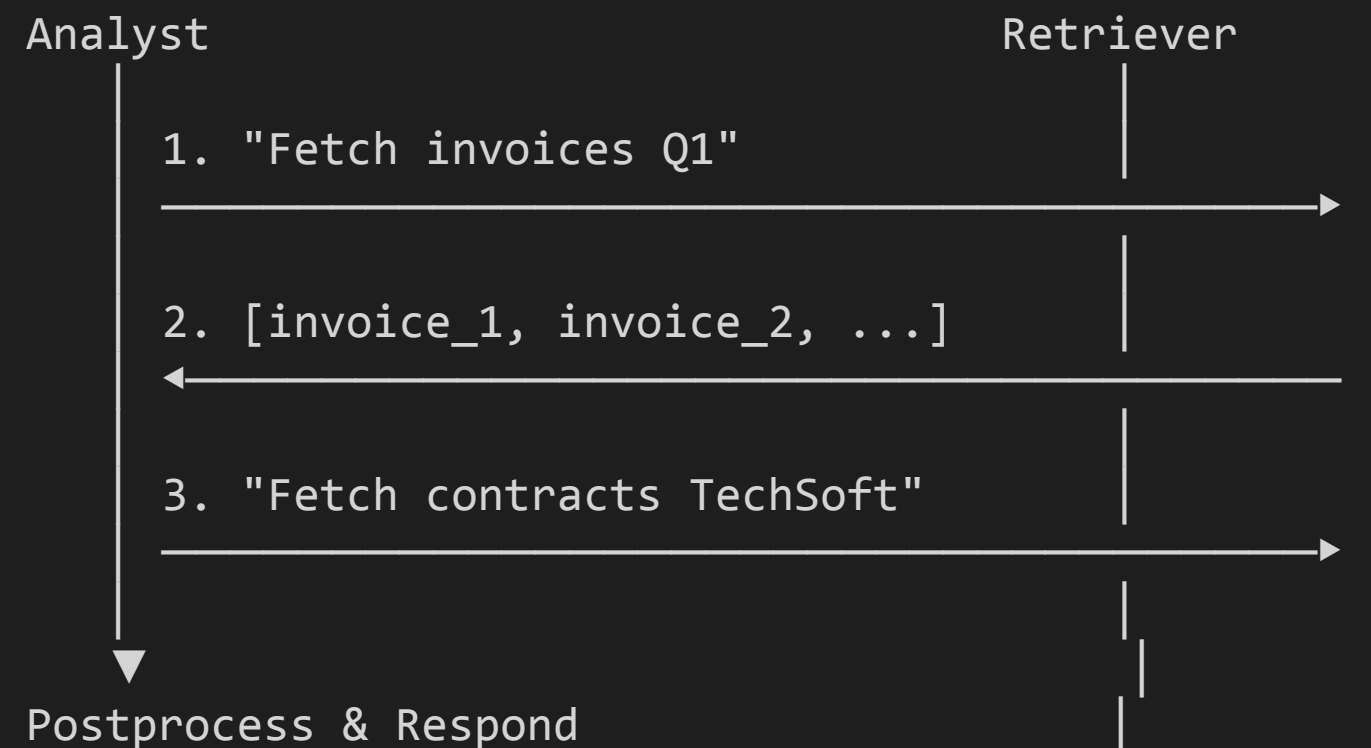
Analyst

orchestrator (CE și CÂND)



Retriever

worker (execută query-uri)



Orchestration în LangGraph

Structura state-ului orchestratorului:

```
1 class RetrievalRequest(BaseModel):
2     intent: str
3     filters: QueryFilters
4     source: Source
5
6
7 class OrchestratorState(TypedDict):
8     """State partajat între Analyst și Retriever."""
9
10    # Input
11    query: str
12
13    # Preprocess output
14    intent: QueryIntent
15    filters: QueryFilters
16
17    # Retriever results – reducer add!
18    raw_data: Annotated[list[dict[str, Any]], operator.add]
19
20    # Postprocess output
21    response: str
22
23    # Control flow
24    phase: Phase
25    retrieval_requests: list[RetrievalRequest]
26    current_request_index: int
27
```

Nodurile Grafului

```
1 def analyst_preprocess(self, state: OrchestratorState) -> dict:
2     prepared = self.analyst.preprocess(state["query"])
3     requests = self._build_retrieval_requests(prepared)
4     return {
5         "intent": prepared["intent"],
6         "filters": prepared["filters"],
7         "retrieval_requests": requests,
8         "current_request_index": 0,
9         "phase": "retrieve",
10    }
11
12 def retriever_node(self, state: OrchestratorState) -> dict:
13     request = state["retrieval_requests"][state["current_request_index"]]
14     data = self.retriever.fetch(request)
15     return {
16         "raw_data": [data], # append via reducer
17         "current_request_index": state["current_request_index"] + 1,
18    }
19
20 def analyst_postprocess(self, state: OrchestratorState) -> dict:
21     prepared = {"intent": state["intent"], "filters": state["filters"]}
22     response = self.analyst.postprocess(state["raw_data"], prepared, state["query"])
23     phase: Phase = "done"
24     return {"response": response, "phase": phase}
25
26 @staticmethod
27 def _build_retrieval_requests(prepared: dict) -> list[RetrievalRequest]:
28     intent: QueryIntent = prepared["intent"]
29     filters: QueryFilters = prepared["filters"]
30     return [
31         RetrievalRequest(
32             intent=intent.intent,
33             filters=filters,
34             source=intent.suggested_source,
35         )
36     ]
```

Routing Logic

3 funcții de routing decid următorul nod:

```
1 def route_after_preprocess(  
2     self, state: OrchestratorState  
3 ) -> Literal["retriever", "postprocess"]:  
4     if state["retrieval_requests"]:  
5         return "retriever"  
6     return "postprocess"  
7  
8 def route_after_retriever(  
9     self, state: OrchestratorState  
10 ) -> Literal["retriever", "postprocess"]:  
11     if state["current_request_index"] < len(state["retrieval_requests"]):  
12         return "retriever" # self-loop  
13     return "postprocess"  
14
```

Construirea Grafului

```
1 def build_orchestrator_graph(analyst: AnalystAgent, retriever: RetrieverAgent):
2     orchestrator = OrchestratorAgent(analyst, retriever)
3
4     # 1. Schema
5     workflow = StateGraph(OrchestratorState)
6
7     # 2. Noduri
8     workflow.add_node("preprocess", orchestrator.analyst_preprocess)
9     workflow.add_node("retriever", orchestrator.retriever_node)
10    workflow.add_node("postprocess", orchestrator.analyst_postprocess)
11
12    # 3. Edge de start
13    workflow.add_edge(START, "preprocess")
14
15    # 4. Edges simple
16    workflow.add_edge("postprocess", END)
17
18    # 5. Conditional edges
19    workflow.add_conditional_edges(
20        "preprocess",
21        orchestrator.route_after_preprocess,
22        {"retriever": "retriever", "postprocess": "postprocess"},
23    )
24    workflow.add_conditional_edges(
25        "retriever",
26        orchestrator.route_after_retriever,
27        {"retriever": "retriever", "postprocess": "postprocess"}, # self-loop
28    )
29
30    # 6. Compile
31    return workflow.compile()
32
```

Vizualizare Graf

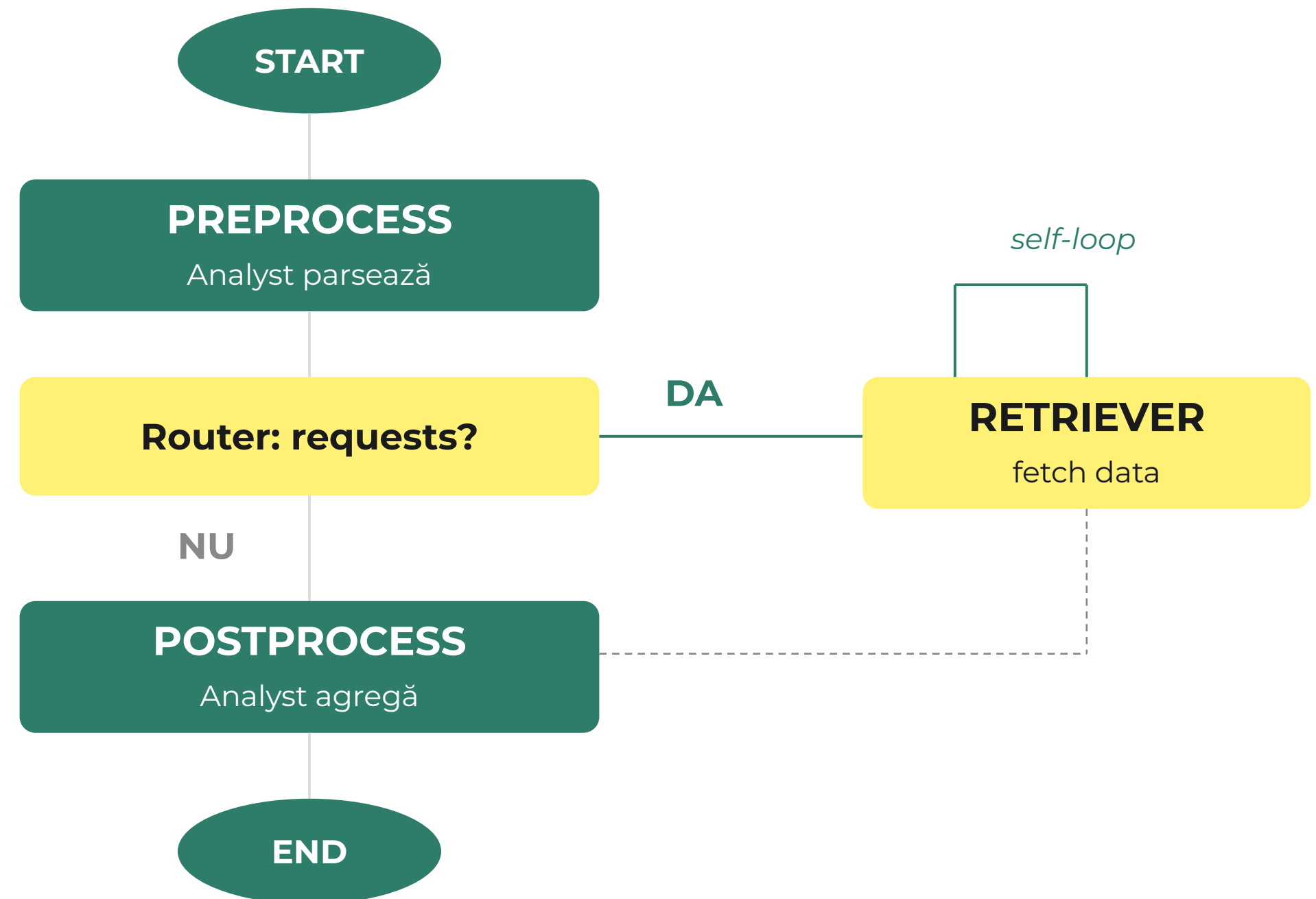
Legendă

 Agent nodes

 Decision / Worker

 Return path

*Self-loop pentru
multiple requests*



preprocess → retriever (loop per request) → postprocess → END

Utilizare Graf

```
1 def main() -> None:
2     # 1. Creăm graful
3     orchestrator = build_orchestrator_graph(AnalystAgent(), RetrieverAgent())
4
5     # 2. Rulăm cu un query
6     result = orchestrator.invoke(
7         {
8             "query": "Care sunt totalurile facturilor TechSoft Q1 2024?",
9             "raw_data": [],
10            "retrieval_requests": [],
11            "current_request_index": 0,
12            "phase": "preprocess",
13        }
14    )
15
16    print(result["response"])
17
18
19 if __name__ == "__main__":
20     main()
21
```

Ce se întâmplă în graf:

1. START → preprocess (Analyst parsează)
2. preprocess → retriever (1 request: facturi TechSoft Q1)
3. retriever → postprocess (am fetch-uit tot)
4. postprocess → END (răspuns formatat)

Multiple Retrieval Requests

Query complex care necesită multiple fetch-uri:

```
query = "Compară totalurile facturilor cu bugetele din contracte pentru Q1"
```

Request 1

```
{
  "type": "invoices",
  "filters": {"period": "Q1"},
  "fields": ["total",
            "supplier"]
}
```

Request 2

```
{
  "type": "contracts",
  "filters": {"period": "Q1"},
  "fields": ["budget",
            "supplier"]
}
```

Graf execută:

preprocess

retriever (invoices)

retriever (contracts)

postprocess

State Accumulation cu Reducers

Problema: fiecare apel Retriever returnează date separate. Soluția: reducer add



```
1 class OrchestratorState(TypedDict):
2     # Cu reducer = se acumulează
3     raw_data: Annotated[list[dict[str, Any]], operator.add]
4
5
6 # Retriever 1 returnează:
7 {"raw_data": [{"invoices": [...]}]}
8
9 # Retriever 2 returnează:
10 {"raw_data": [{"contracts": [...]}]}
11
```

Retriever 1

Retriever 2

add reducer

```
raw_data: [
  {invoices},
  {contracts}
]
```

Passing Context între Agenți

Cum Analyst transmite context la Retriever:

```
1 class OrchestratorAgent:
2     def __init__(self, analyst: AnalystAgent, retriever: RetrieverAgent):
3         self.analyst = analyst
4         self.retriever = retriever
5
6     def analyst_preprocess(self, state: OrchestratorState) -> dict:
7         request = RetrievalRequest(
8             intent=state["intent"].intent,
9             filters=state["filters"],
10            source=state["intent"].suggested_source,
11            context=RequestContext(
12                original_query=state["query"],
13                expected_fields=self._detect_expected_fields(state["query"]),
14                result_limit=100,
15            ),
16        )
17        return {"retrieval_requests": [request]}
18
19    def retriever_node(self, state: OrchestratorState) -> dict:
20        request = state["retrieval_requests"][state["current_request_index"]]
21        data = self.retriever.fetch(request) # folosește request.context intern
22        return {"raw_data": [data]}
23
24    def _detect_expected_fields(self, query: str) -> list[str]:
25        ...
26
```

Parallel Retrieval (Optional)

Pentru performanță: fetch-uri în paralel cu Send API din LangGraph

```
1 from langgraph.graph import Send
2
3
4 class RetrieverNodeState(TypedDict):
5     request: RetrievalRequest
6     request_index: int
7
8
9 class OrchestratorAgent:
10     # __init__, alte noduri și routere ...
11
12     def route_to_parallel(self, state: OrchestratorState) -> list[Send]:
13         """Trimite toate request-urile în paralel."""
14         return [
15             Send("retriever", {"request": request, "request_index": i})
16             for i, request in enumerate(state["retrieval_requests"])
17         ]
18
19     def retriever_node(self, state: RetrieverNodeState) -> dict:
20         data = self.retriever.fetch(state["request"])
21         return {"raw_data": [data]} # append via reducer
22
```

Send API permite map-reduce pattern în LangGraph



Rezultat:

Toate fetch-urile rulează
SIMULTAN, nu secvențial

3 secvențial



3 paralel



Logging + Observability

Logging

```
1 from langfuse import observe
2 logger = logging.getLogger(__name__)
3
4
5 class OrchestratorAgent:
6     # __init__ ...
7
8     @observe(name="preprocess")
9     def analyst_preprocess(self, state: OrchestratorState) -> dict:
10         logger.info("Preprocess started: query=%r", state["query"][:80])
11         prepared = self.analyst.preprocess(state["query"])
12         logger.info("Preprocess done: intent=%s", prepared["intent"].intent)
13         return prepared
14
15     @observe(name="retriever")
16     def retriever_node(self, state: RetrieverNodeState) -> dict:
17         idx = state["request_index"]
18         logger.info("Retriever fetch: index=%d source=%s", idx, state["request"].source)
19         data = self.retriever.fetch(state["request"])
20         logger.debug("Retriever result: index=%d size=%d", idx, len(data))
21         return {"raw_data": [data]}
22
```

În Langfuse vezi:

- ✓ Timeline cu toate nodurile
- ✓ Durata fiecărui pas
- ✓ Input/output per nod

Timeout și Cancellation

Protecție împotriva execuțiilor blocate sau prea lungi:



```
1 class OrchestratorTimeout(Exception):
2     """Orchestrator nu s-a încheiat în intervalul alocat."""
3
4     def __init__(self, timeout: float):
5         super().__init__(f"Orchestrator timeout after {timeout:.1f}s")
6         self.timeout = timeout
7
8
9 async def run_with_timeout(orchestrator, state: dict, timeout: float = 30.0) -> dict[str, Any]:
10     """Rulează orchestratorul cu timeout. Aruncă OrchestratorTimeout la depășire."""
11     try:
12         return await asyncio.wait_for(orchestrator.ainvoke(state), timeout=timeout)
13     except asyncio.TimeoutError as e:
14         logger.warning("Orchestrator timeout after %.1fs", timeout)
15         raise OrchestratorTimeout(timeout) from e
16
```

Folosește `ainvoke()` (async) în loc de `invoke()` pentru timeout suport

Exemplu Complet End-to-End

```
1  # 1. Setup
2  orchestrator = build_orchestrator_graph()
3
4  # 2. Query complex
5  query = """
6  Vreau un raport cu:
7  - Totalurile facturilor TechSoft Q1
8  - Comparat cu bugetul din contract
9  - Diferențele > 10%
10 """
11
12 # 3. Invocare
13 result = orchestrator.invoke({
14     "query": query,
15     "raw_data": [],
16     "retrieval_requests": [],
17     "current_request_index": 0,
18 })
19
```

Output

Raport TechSoft Q1 2024:

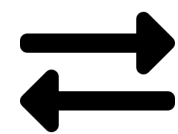
Facturi vs Buget:

- Total facturi: 16,700 RON
- Buget contract: 15,000 RON
- Diferență: +1,700 RON (+11.3%)

 **Alertă: diferența depășește pragul de 10%**

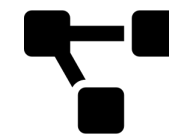
Surse: 3 facturi, 1 contract

Sumar — Secțiunea 4



Orchestration

Analyst decide CE și CÂND
Retriever execută fetch-urile



LangGraph Graph

StateGraph cu state partajat
3 noduri + conditional routing
Self-loop pentru multi-fetch



State Management

Reducer add pentru acumulare
Context passing între agenți
Reducers vs replace



Observability

Logging la fiecare nod
LangSmith tracing automat
Timeout/cancellation

Next: Secțiunea 5 — Error Handling Between Agents



Secțiunea 5

Error Handling

Retry, fallback și graceful degradation între agenți

De Ce E Diferit în Multi-Agent?

Single Agent — simplu:

```
try:
    result = agent.process(query)
except Exception as e:
    return f"Eroare: {e}"r
```

Multi-Agent — întrebări:

- ⚠ În ce agent a apărut eroarea?
- ⚠ Cine decide ce se întâmplă?
- ⚠ Cum propagăm eroarea?
- ⚠ Retry local sau global?

User Query → Analyst → Retriever → ?

ANALYST



Eroare!

RETRIEVER

- *Analyst retry?*
- *Retriever retry?*
- *Fallback?*
- *Abort?*

→ *Răspunsul depinde de tipul și severitatea erorii*

Tipuri de Erori în Document Analyst

Eroare	Unde apare	Severitate	Acțiune
Query parse error	Analyst preprocess	LOW	Cere clarificare
Invalid filters	Analyst preprocess	LOW	Adjust și retry
DB connection	Retriever	HIGH	Retry sau fallback
Empty results	Retriever	LOW	Raportează, nu e eroare
Timeout	Retriever	MEDIUM	Retry cu limite
Schema mismatch	Analyst postprocess	MEDIUM	Partial result
LLM rate limit	Oricare	MEDIUM	Backoff și retry

Nu toate erorile necesită abort — clasificarea ghidează strategia

Temă: Data Reader Agent cu LangGraph

Construiește un agent cu state tipat, noduri și retry/fallback — fundația pentru orchestrarea din L6

Flow așteptat:

```

1
2 # graph.py – state → nodes → conditional edges
3 class DataReaderState(BaseModel):
4     query: str
5     source: str = "" # rag|csv|sql
6     retry_count: int = 0
7     result: dict | None = None
8
9 def route(s) -> Literal[...]:
10     if s.error and s.retry_count < 3:
11         return "retry"
12     return "end"
13
14 g = StateGraph(DataReaderState)
15 g.add_conditional_edges("execute", route)
16 app = g.compile()
17

```

Ce trebuie făcut:

- State Schema tipat (L5)**
 - TypedDict / Pydantic cu query, source, retry_count, result, error
 - Reducers pentru câmpuri agregate (ex: sources_tried)
- Noduri (L5)**
 - parse_query → decide_source (LLM alege rag/csv/sql)
 - generate_query → execute_query → check_result
- Conditional Edges (L5)**
 - Routing pe baza sursei alese de LLM
 - add_conditional_edges pentru ramificare
- Retry + Fallback (L5)**
 - Loop de retry la eroare (max_retries)
 - Demo: agent răspunde din RAG, CSV sau SQL

Temă: Multi-Agent Orchestration

Extinde Data Reader-ul din L5 într-un sistem multi-agent cu Supervisor, shared state și error handling

Flow așteptat:

```
1 # orchestrator.py – supervisor → workers
2 class OrchestratorState(BaseModel):
3     query: str
4     selected_agents: list[str]
5     agent_results: dict[str, AgentResult]
6
7 def supervisor(s) -> dict:
8     # LLM alege ce agenți rulează
9     agents = choose(s.query)
10    return {"selected_agents": agents}
11
12 g.add_node("supervisor", supervisor)
13 g.add_node("rag", rag_worker)
14 g.add_node("aggregate", aggregate)
15 app.invoke({"query": q})
16
```

Ce trebuie făcut:

1

Supervisor Pattern (L6)

- Agent central care alege workers (rag/sql/csv)
- Wrap agentul din L5 ca un worker

2

Shared State + Comunicare (L6)

- TypedDict cu reducers pentru agent_results
- Message passing între supervisor și workers

3

Analyst: pre/postprocess (L6)

- Preprocess: detect_intent, extract_filters
- Postprocess: aggregate_results, format_response

4

Error Handling + Bonus (L6)

- Retry, fallback,